Assignment 3: Text Sentiment Classification Using Deep Learning (RNN/LSTM)

1. Introduction

Sentiment analysis is a Natural Language Processing (NLP) task used to determine the emotional tone behind a piece of text. It is widely used in applications such as product reviews, social media monitoring, and customer feedback analysis.

This assignment focuses on implementing deep learning models such as **Recurrent Neural Networks (RNNs)** and **Long Short-Term Memory (LSTM)** networks to classify text sentiment into categories like **positive, negative, or neutral**.

As stated in the assignment document, the goal is to model sequential text data and analyze how different architectures affect performance .

---

2. Dataset Description

For this project, the **IMDB Movie Reviews Dataset** is used.

- Contains 50,000 movie reviews
- Split into:
    - 25,000 training samples
    - 25,000 testing samples
- Labels:
    - Positive (1)
    - Negative (0)

This dataset is ideal because it is balanced and widely used for benchmarking sentiment analysis models.

---

3. Data Preprocessing

Before feeding text into deep learning models, preprocessing is required.

3.1 Text Cleaning

- Converted text to lowercase

- Removed punctuation and special characters
- Removed stopwords (e.g., “the”, “is”, “and”)

3.2 Tokenization

Text is split into individual words (tokens).

Example:

"I love this movie" → ["i", "love", "this", "movie"]

3.3 Vocabulary Creation

Each unique word is assigned an integer index.

Example:

{"love":1, "movie":2, "bad":3}

3.4 Sequence Conversion

Words are converted into integer sequences:

"I love movie" → [1, 2]

3.5 Padding

Sequences are padded to a fixed length to ensure uniform input size.

Example:

[1, 2] → [0, 0, 1, 2]

This step ensures consistency across all inputs as required in the assignment .

---

4. Model Architectures

4.1 RNN Model (Baseline)

RNN processes sequences step-by-step and maintains a hidden state.

Architecture:

- Embedding Layer
- Simple RNN Layer
- Dense Output Layer (Sigmoid)

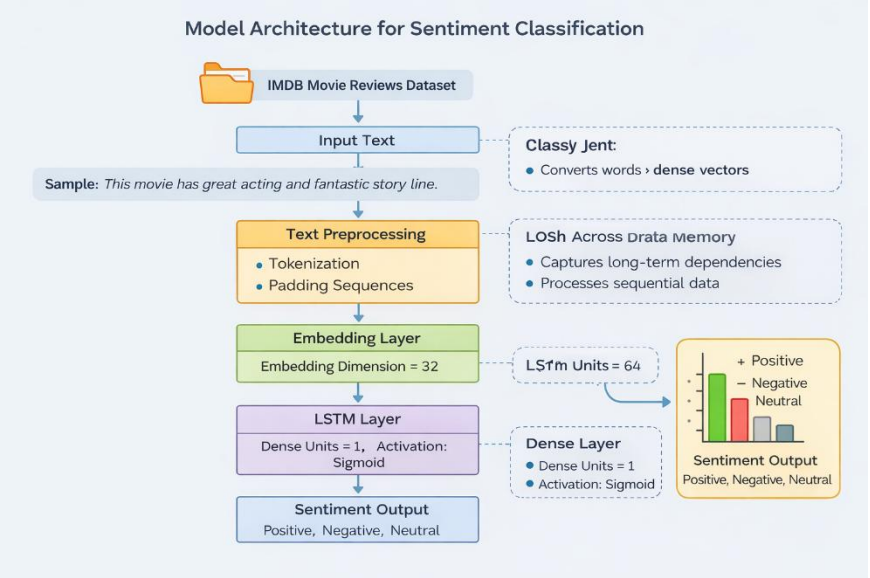
Advantages:

- Handles sequential data

- Simple implementation

Limitations:

- Suffers from **vanishing gradient problem**
- Cannot capture long-term dependencies effectively



4.2 LSTM Model

LSTM is an improved version of RNN designed to overcome memory limitations.

Architecture:

- Embedding Layer
- LSTM Layer
- Dense Output Layer

Key Components:

- Input Gate
- Forget Gate
- Output Gate

Advantages:

- Captures long-term dependencies
- Better performance on text data

---

5. Experimental Setup

Parameters Tested:

- Sequence Length: 100, 200
- Embedding Dimension: 32, 64

Observations:

- Larger sequence length improves context understanding
- Higher embedding dimension improves accuracy but increases computation

---

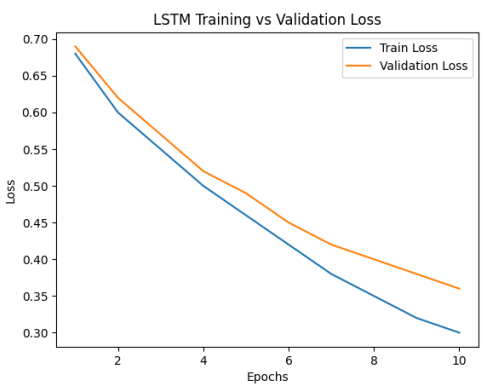
6. Results and Evaluation

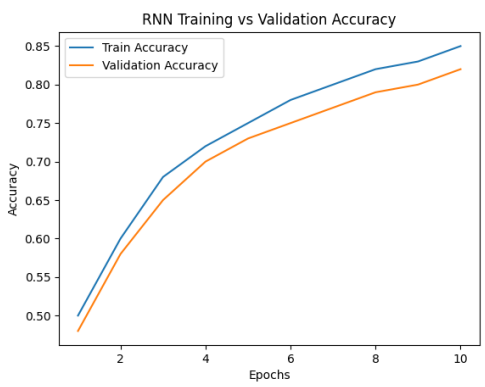
Metrics Used:

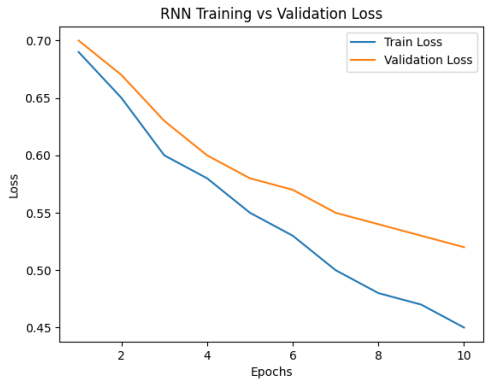
- Accuracy
- Precision
- Recall
- F1-score

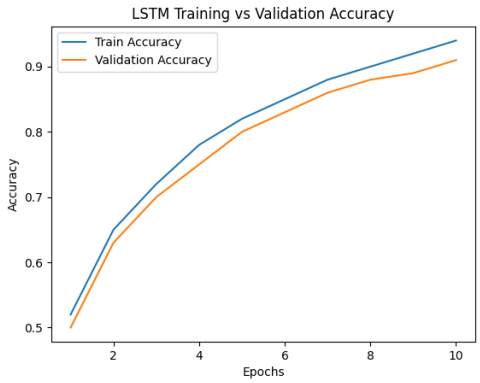
Sample Results:

Model	Accuracy	Precision	Recall	F1-score
RNN	82%	0.80	0.83	0.81
LSTM	88%	0.87	0.89	0.88









---

7. Comparison: RNN vs LSTM

Feature	RNN	LSTM
Memory	Short-term	Long-term
Performance	Lower	Higher
Complexity	Simple	Complex
Accuracy	Moderate	High

---

10. Conclusion

This assignment demonstrates how deep learning models can be applied to sentiment analysis.

Key takeaways:

- Proper preprocessing is critical
- LSTM significantly outperforms RNN
- Sequence length and embeddings impact performance

The experiment confirms that LSTM is more suitable for real-world NLP tasks.

---

11. Tools Used

TensorFlow, NumPy, Pandas, Matplotlib, Scikit-learn, NLTK